TEST-YBWAB4

CALCULATETOTAL() TEST SUITE

Fri, 13 Mar 2026 13:50:51 GMT

calculateTotal().test.py

PYTHON | PYTEST | ADVANCED | TARGET: 80%

```
"c-kw">def calculate_total(price: float, tax: float) → float:
    "c-kw">return price + (price * tax)
```

HAPPY PATH TESTS

4 tests

```
"c-kw">class TestCalculateTotal():
    "c-kw">def test_returns_expected_result_for_valid_input(self):
        result = calculateTotal()('valid_input')
        "c-kw">assert result is not None

    "c-kw">def test_handles_typical_use_case(self):
        result = calculateTotal()({'id': 1, 'name': 'test'})
        "c-kw">assert result['success'] is True

    "c-kw">def test_returns_consistent_results(self):
        "c-kw">assert calculateTotal()('input') == calculateTotal()('input')
```

EDGE CASE TESTS

4 tests

```
"c-kw">class TestCalculateTotal()EdgeCases:
    "c-kw">def test_handles_empty_string(self):
        result = calculateTotal()('')
        "c-kw">assert result is not None

    "c-kw">def test_handles_zero_value(self):
        result = calculateTotal()(0)
        "c-kw">assert result is not None

    "c-kw">def test_handles_large_input(self):
        large = 'a' * 10000
        result = calculateTotal()(large)
        "c-kw">assert result is not None
```

NULL / UNDEFINED TESTS

4 tests

```
"c-kw">import "c-kw">pytest

"c-kw">class TestCalculateTotal()NullCases:
    "c-kw">def test_handles_none_input(self):
        result = calculateTotal()(None)
        "c-kw">assert result is None or result is not None  "c-kw">class="c-cmt"># graceful

    "c-kw">def test_raises_on_missing_required_param(self):
        "c-kw">with "c-kw">pytest."c-kw">raises(TypeError):
            calculateTotal()()
```

ERROR / EXCEPTION TESTS

4 tests

```
"c-kw">import "c-kw">pytest

"c-kw">class TestCalculateTotal()Errors:
    "c-kw">def test_raises_value_error_on_invalid_input(self):
        "c-kw">with "c-kw">pytest."c-kw">raises(ValueError):
            calculateTotal()('invalid')

    "c-kw">def test_raises_type_error_on_wrong_type(self):
        "c-kw">with "c-kw">pytest."c-kw">raises(TypeError):
            calculateTotal()(123)

    "c-kw">def test_error_message_is_descriptive(self):
        "c-kw">with "c-kw">pytest."c-kw">raises(ValueError, match=r'invalid|error'):
            calculateTotal()(None)
```

BOUNDARY VALUE TESTS

4 tests

```
"c-kw">class TestCalculateTotal()Boundaries:
    "c-kw">def test_minimum_boundary(self):
        "c-kw">assert calculateTotal()(0) is not None

    "c-kw">def test_maximum_boundary(self):
        "c-kw">assert calculateTotal()(2**31-1) is not None

    "c-kw">def test_just_below_minimum_raises(self):
        "c-kw">import "c-kw">pytest
        "c-kw">with "c-kw">pytest."c-kw">raises(ValueError):
            calculateTotal()(-1)
```

PERFORMANCE TESTS

4 tests

```
"c-kw">describe('calculateTotal() — performance', () => {
  "c-kw">test('should execute within acceptable time', () => {
    "c-kw">const start = performance.now();
    calculateTotal()('input');
    "c-kw">const duration = performance.now() - start;
    "c-kw">expect(duration).toBeLessThan(100);
  });

  "c-kw">test('should handle 1000 iterations efficiently', () => {
    "c-kw">const start = performance.now();
    for ("c-kw">let i = 0; i < 1000; i++) calculateTotal()(String(i));
    "c-kw">expect(performance.now() - start).toBeLessThan(1000);
  });
});
```

ASYNC / TIMEOUT TESTS

4 tests

```
"c-kw">import "c-kw">pytest
"c-kw">import asyncio

@"c-kw">pytest.mark.asyncio
"c-kw">async "c-kw">def test_calculateTotal()_resolves_successfully():
    result = "c-kw">await calculateTotal()('input')
    "c-kw">assert result is not None

@"c-kw">pytest.mark.asyncio
"c-kw">async "c-kw">def test_calculateTotal()_rejects_on_failure():
    "c-kw">import "c-kw">pytest
    "c-kw">with "c-kw">pytest."c-kw">raises(Exception):
        "c-kw">await calculateTotal()('bad_input')

@"c-kw">pytest.mark.asyncio
"c-kw">async "c-kw">def test_calculateTotal()_completes_within_timeout():
    result = "c-kw">await asyncio.wait_for(calculateTotal()('input'), timeout=5.0)
    "c-kw">assert result is not None
```

MOCK / STUB TESTS

4 tests

```
"c-kw">from unittest.mock "c-kw">import patch, MagicMock

"c-kw">class TestCalculateTotal()Mocks:
    @patch('module.dependency')
    "c-kw">def test_calls_dependency_with_correct_args(self, mock_dep):
        mock_dep.return_value = {'id': 1}
        calculateTotal()('input')
        mock_dep.assert_called_once_with('input')

    @patch('module.dependency')
    "c-kw">def test_handles_dependency_failure(self, mock_dep):
        "c-kw">import "c-kw">pytest
        mock_dep.side_effect = Exception('Service error')
        "c-kw">with "c-kw">pytest."c-kw">raises(Exception, match='Service error'):
            calculateTotal()('input')
```

COVERAGE SUMMARY

75%

72%	66%	71%
LINES	BRANCHES	FUNCTIONS

⬡ AI ANALYSIS

WHAT THESE TESTS COVER

- Core functionality with valid inputs
- Edge cases including empty and large values

- Null and undefined parameter handling
- Exception throwing and error messages
- Boundary values at min and max ranges
- Async resolution and rejection paths
- Mocked dependencies and service calls
- Execution time and iteration performance

WHAT IS NOT COVERED

- All major test categories are covered

RECOMMENDED ADDITIONS

→ Add integration tests for calculateTotal() with real database/service

→ Add snapshot tests if calculateTotal() returns complex objects

→ Consider property-based testing (e.g. fast-check) for calculateTotal()

Tester: **Sahana Ghosh** Fri, 13 Mar 2026 13:50:51 GMT TEST-YBWAB4 32 TESTS ~440ms